Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

mongosh
use aggregation_lab
db.orders.insertMany([
{ _id: 1, productId: 101, quantity: 2, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-15T00:00:00Z") },
{ _id: 2, productId: 102, quantity: 1, price: 50.00, customerId: "B456", orderDate: ISODate("2024-01-20T00:00:00Z") },
{ _id: 3, productId: 101, quantity: 3, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-25T00:00:00Z") },
{ _id: 4, productId: 103, quantity: 1, price: 75.00, customerId: "C789", orderDate: ISODate("2024-02-01T00:00:00Z") },
{ _id: 5, productId: 102, quantity: 2, price: 50.00, customerId: "B456", orderDate: ISODate("2024-02-10T00:00:00Z") }
])

db.products.insertMany([
{ _id: 101, name: "Widget A", category: "Widgets" },
{ _id: 102, name: "Widget B", category: "Widgets" },
{ _id: 103, name: "Gadget X", category: "Gadgets" }
])

```
test> use aggregation_lab
switched to db aggregation_lab
aggregation_lab> db.orders.insertMany([
aggregation_lab> db.orders.insertMany([
aggregation_lab> db.orders.insertMany([
aggregation_lab> db.orders.insertMany([
| { _id: 1, productId: 101, quantity: 2, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-15T00:00:00Z") },
| { _id: 2, productId: 102, quantity: 1, price: 50.00, customerId: "B456", orderDate: ISODate("2024-01-20T00:00:00Z") },
| { _id: 3, productId: 101, quantity: 3, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-25T00:00:00Z") },
| { _id: 4, productId: 103, quantity: 1, price: 75.00, customerId: "C789", orderDate: ISODate("2024-02-01T00:00:00Z") },

| { _id: 5, productId: 102, quantity: 2, price: 50.00, customerId: "B456", orderDate: ISODate("2024-02-10T00:00:00Z") }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
aggregation_lab> db.products.insertMany([
| { _id: 101, name: "Widget A", category: "Widgets" },
| { _id: 102, name: "Widget B", category: "Widgets" },
| { _id: 103, name: "Gadget X", category: "Gadgets" }
| ])
|
{ acknowledged: true, insertedIds: { '0': 101, '1': 102, '2': 103 } }
aggregation_lab>
```

Step 1: Basic $group Stage: Calculate Total Order Count
Description: This acts like a simple counter. By setting _id: null, you tell MongoDB to ignore categories and put every single document into one big bucket to count them.

db.orders.aggregate([
{
$group: {
_id: null, // Group all documents into a single group
totalOrders: { $sum: 1 } // Count each document
}

}
])

```
aggregation_lab> db.products.insertMany([
| { _id: 101, name: "Widget A", category: "Widgets" },
| { _id: 102, name: "Widget B", category: "Widgets" },
| { _id: 103, name: "Gadget X", category: "Gadgets" }
| ])
|
{ acknowledged: true, insertedIds: { '0': 101, '1': 102, '2': 103 } }
aggregation_lab> db.orders.aggregate([
| {
| $group: {
| _id: null, // Group all documents into a single group
| totalOrders: { $sum: 1 } // Count each document
| }
| }
| ])
|
[ { _id: null, totalOrders: 5 } ]
aggregation_lab>
```

Step 2: $group with $sum and Grouping by productid

Description: Instead of one big bucket, this creates a separate bucket for every unique productId. It then adds up the quantity values for each specific product.

db.orders.aggregate([
{
$group: {
_id: "$productId", // Group by productId
totalQuantity: { $sum: "$quantity" } // Sum the quantity for each product
}
}
])

```
aggregation_lab> db.orders.aggregate([
| {
| $group: {
| _id: "$productId", // Group by productId
| totalQuantity: { $sum: "$quantity" } // Sum the quantity for each product
| }
| }
| ])
|
[
  { _id: 102, totalQuantity: 3 },
  { _id: 101, totalQuantity: 5 },
  { _id: 103, totalQuantity: 1 }
]
aggregation_lab>
```

Step 3: $group with Multiple Accumulators: Total Quantity and Average Price

Description: This works just like Step 2, but it performs two math tasks at once. It calculates the total volume (sum) and the average price (avg) for each product bucket.

db.orders.aggregate([
{
$group: {
_id: "$productId",
totalQuantity: { $sum: "$quantity" },

averagePrice: { $avg: "$price" }
}
}
])

```
aggregation_lab> db.orders.aggregate([
| {
| $group: {
| _id: "$productId",
| totalQuantity: { $sum: "$quantity" },
| averagePrice: { $avg: "$price" }
| }
| }
| ])
|
[
  { _id: 103, totalQuantity: 1, averagePrice: 75 },
  { _id: 101, totalQuantity: 5, averagePrice: 25 },
  { _id: 102, totalQuantity: 3, averagePrice: 50 }
]
aggregation_lab>
```

Step 4: Basic $lookup Stage: Joining orders and products Collections
Description: This is MongoDB's version of a SQL JOIN. It looks at the productId in orders and finds the matching document in the products collection, attaching the info as a new array called productInfo.

db.orders.aggregate([
{
$lookup: {
from: "products", // The collection to join with
localField: "productId", // The field from the input documents
foreignField: "_id", // The field from the "from" collection
as: "productInfo" // The name of the new array field
}
}
])

```
aggregation_lab> db.orders.aggregate([
| {
| $lookup: {
| from: "products", // The collection to join with
| localField: "productId", // The field from the input documents
| foreignField: "_id", // The field from the "from" collection
| as: "productInfo" // The name of the new array field
| }
| }
| ])
|
[
  {
    _id: 1,
    productId: 101,
    quantity: 2,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-15T00:00:00.000Z'),
    productInfo: [ { _id: 101, name: 'Widget A', category: 'Widgets' } ]
  },
  {
    _id: 2,
    productId: 102,
    quantity: 1,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-01-20T00:00:00.000Z'),
    productInfo: [ { _id: 102, name: 'Widget B', category: 'Widgets' } ]
  },
  {
    _id: 3,
    productId: 101,
    quantity: 3,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-25T00:00:00.000Z'),
    productInfo: [ { _id: 101, name: 'Widget A', category: 'Widgets' } ]
  },
  {
    _id: 4,
    productId: 103,
    quantity: 1,
    price: 75,
    customerId: 'C789',
    orderDate: ISODate('2024-02-01T00:00:00.000Z'),
    productInfo: [ { _id: 103, name: 'Gadget X', category: 'Gadgets' } ]
  },
  {
    _id: 5,
    productId: 102,
    quantity: 2,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-02-10T00:00:00.000Z'),
    productInfo: [ { _id: 102, name: 'Widget B', category: 'Widgets' } ]
  }
]
aggregation_lab>
```

Step 5: Combining $lookup and $unwind Stages

Description: By default, $lookup creates an array (even if there is only one match). $unwind flattens that array, turning it into a regular object so it is easier to access the fields inside it.

db.orders.aggregate([

```
{
$lookup: {
from: "products",
localField: "productId",
foreignField: "_id",
as: "productInfo"
}
},
{
$unwind: "$productInfo"
}
])
```

```
aggregation_lab> db.orders.aggregate([
| {
| $lookup: {
| from: "products",
| localField: "productId",
| foreignField: "_id",
| as: "productInfo"
| }
| },
| {
| $unwind: "$productInfo"
| }
| ])
|
[
  {
    _id: 1,
    productId: 101,
    quantity: 2,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-15T00:00:00.000Z'),
    productInfo: { _id: 101, name: 'Widget A', category: 'Widgets' }
  },
  {
    _id: 2,
    productId: 102,
    quantity: 1,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-01-20T00:00:00.000Z'),
    productInfo: { _id: 102, name: 'Widget B', category: 'Widgets' }
  },
  {
    _id: 3,
    productId: 101,
    quantity: 3,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-25T00:00:00.000Z'),
    productInfo: { _id: 101, name: 'Widget A', category: 'Widgets' }
  },
  {
    _id: 4,
    productId: 103,
    quantity: 1,
    price: 75,
    customerId: 'C789',
    orderDate: ISODate('2024-02-01T00:00:00.000Z'),
    productInfo: { _id: 103, name: 'Gadget X', category: 'Gadgets' }
  },
  {
    _id: 5,
    productId: 102,
    quantity: 2,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-02-10T00:00:00.000Z'),
    productInfo: { _id: 102, name: 'Widget B', category: 'Widgets' }
  }
]
aggregation_lab>
```

Step 6: Combining $lookup and $group to Calculate Total Sales by Category

Description: This is the "Grand Finale." It connects the two tables, flattens the result, and then groups the data by Category (which lives in the joined product table) to calculate the total revenue ($Price x Quantity$).

```
db.orders.aggregate([
{
$lookup: {
from: "products",
localField: "productId",
foreignField: "_id",
as: "productInfo"
}
},
{
$unwind: "$productInfo"
},
{
$group: {
_id: "$productInfo.category",
totalSales: { $sum: { $multiply: ["$quantity", "$price"] } }
}
}
])
```